

情報科学演習 D

課題 1

担当教員 : 梶井 晃基
提出者 : 鶴見 俊介
所属/学年 : 基礎工学部 情報科学科 3 年 計算機科学コース
学籍番号 : 09B23057
電子メール : u299151f@ecs.osaka-u.ac.jp

提出日 : 2025 年 10 月 15 日

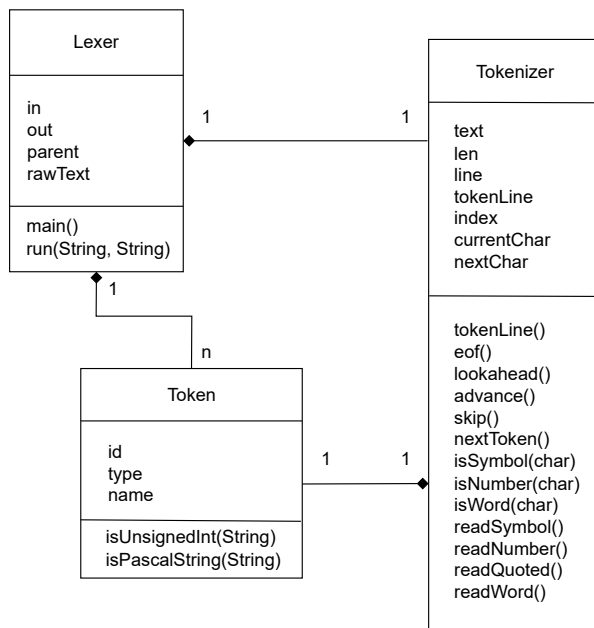
締切日 : 2025 年 10 月 17 日

1 システムの仕様

Pascal 風言語のソースから字句解析によりトークン列を生成し、トークン名・種別・ID・行番号を出力する。Lexer.run(String,String) により、第一引数で入力.pas、第二引数で出力.ts を指定する。出力は各行をタブ区切りで「[トークン名][トークン種][トークン ID][行番号]」とする。処理が正常終了した場合は "OK", 入力ファイル未発見時は "File not found" を返す。

2 課題達成の方針と設計

2.1 各機能の分担



(a) クラス図

クラス名	役割
Lexer	入出力と制御を担当する。 .pas ファイルを読み込み Tokenizer を駆動し、トークン列を取得して.ts に出力する。終了時に"OK"か"File not found"を出力する。
Tokenizer	.pas ファイルの文字列からトークンを切り出して、Token を生成する。同時に行番号の管理も行う。
Token	トークン ID, トークン種, トークン名を保持する不変オブジェクト。渡されたトークンを判別する。

(b) 各クラスの役割

図 1: クラス図と各クラスの役割の対応

字句解析は、図 1 のように 3 つのクラスに機能を分担して実装する。図 1a は各クラス間の関係と、各クラスの持つフィールドとメソッドを示しており、図 1b は各クラスの役割を示している。これらのオブジェクトによって、以下の流れで機能を実現する。

1. Lexer が入力ファイルを読み取り、入力ファイル内の全文字列を取得する。そして、Tokenizer を生成し、取得した全文字列を渡す。
2. Tokenizer が文字を分析し、トークンごとに分割する。そして、Token を生成して分割したトークン文字列を渡す。
3. Token は渡されたトークンの ID とトークン種を取得する。
4. Lexer は Token からトークン名、トークン種、トークン ID を、Tokenizer から行番号を取得し、出力ファイルに書き出す。まだ解析すべき文字が残っているなら 2 へ戻る。

2.2 トークンの分割

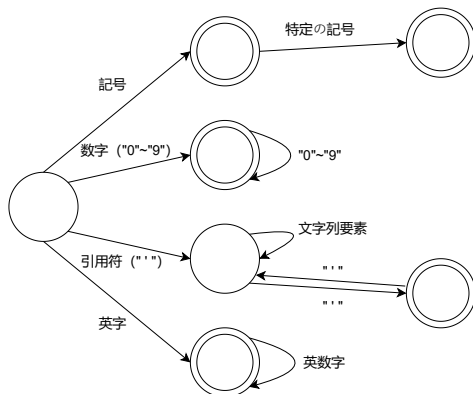


図 2: 決定性有限オートマトン

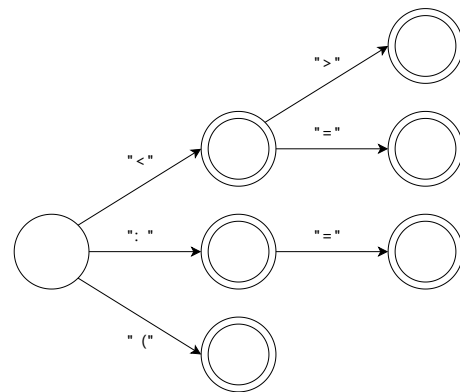


図 3: 決定性有限オートマトン（記号の例）

Tokenizer は図 2 の決定性有限オートマトンで終端を判定する。先頭文字により「記号・数字・文字列・英数字」の 4 系統へ分岐し、系統ごとに終了条件を定義する。数字は非数字が現れた時点で確定し、文字列は引用符で開始し ' ' を内部の一重引用符として扱う。記号は最長一致を基本とし、必要時のみ 2 文字目を先読みして図 3 の規則に従い確定する。

2.3 トークンの判別

Tokenizer によって切り出されたトークンにおいて、トークン種とトークン ID を調べる必要がある。そのために、トークンに関する情報を保持しているのが **Token** オブジェクトである。**Token** は、生成時にトークン名の文字列を渡されるため、そのトークン名がどれであるかを文字列一致で調べる。45 通りのトークン種のうち、予約語や記号などの文字列に一致しなかったトークンは、識別子・符号なし整数・文字列のいずれかとなる。この判別は、トークンがどの文字で構成されているかを調べればよい。このようにして判別された各トークンは、**Token** が保持しているトークン種とトークン ID を取得する。

3 実装プログラム

3.1 Lexer

Lexer は、入力ファイルを **in**、出力ファイルを **out** に保持し、**in** からファイル内の全文字列を取得し **rawText** に代入する。この **rawText** を **Tokenizer** に渡してオブジェクトを生成し、トークンを切り出す。トークンの切り出しは、**Token** 型の **Tokenizer.nextToken()** メソッドで行い、名前、種類、ID、行番号を取得する。これらを **out** に書き出すことを繰り返し、処理が終了したら "OK" を返す。

3.2 Tokenizer

Tokenizer は、**Lexer** からファイル内の全文字列を渡されて生成され、そのファイル内文字列をトークンに分割する機能を持つ。トークンへの分割は、このオブジェクトの **Token** 型 **nextToken()** メソッドを呼び出すことで実現する。プログラム 1 は、このメソッドを簡易的に記したものである。

Tokenizer は、**currentChar** で現在参照している文字を保持し、**nextChar** でその次の文字を保持して、決定性有限オートマトン（図 2）を表現している。また、読み出した文字が改行文字であるとき、**line** フィー

ルドをインクリメントして行番号を管理する。nextToken() メソッドが呼び出されると、まず skip() メソッドによって、currentChar が何らかの文字を保持するまで空白や改行を読み飛ばす。次に、先頭の文字 (currentChar) が、記号・数字・引用符・英字のどれに分類されるかを調べ、それぞれ適切な処理へ分岐する。

例えば、先頭の文字が数字であった場合は、isNumber(char) メソッドが true を返し、readNumber() メソッドを呼び出す。readNumber() メソッドは、currentChar と nextChar を更新しながらそれ以降の文字を順に調べていく。このとき、読み出した文字を StringBuilder に格納していき、トークン文字列を作成する。読み出した文字が数字でないためにトークンの終端を判定したら、Token オブジェクトを生成しトークン文字列を渡して、処理を終了する。

このような処理を記号・引用符・英字の場合も繰り返すことで、すべてのトークンに分割する。なお、eof() メソッドが true を返したとき、ファイルの読み取りが終了したとして null を返すようにしている。

プログラム 1: Token.java

```
1 public class Tokenizer {
2     public Token nextToken() {
3         skip();
4         if (eof()) return null;
5         if (isSymbol(currentChar)) {
6             return readSymbol();
7         } else if (isNumber(currentChar)) {
8             return readNumber();
9         } else if (currentChar == '\\') {
10            return readQuoted();
11        } else {
12            return readWord();
13        }
14    }
15 }
```

3.3 Token

Token は、Tokenizer からトークンを渡されて生成され、コンストラクタ内で switch 文によって文字列が一致するか検証しトークンを調べる。その様子を簡易的に書き出したものがプログラム 2 である。渡されたトークンが予約語や記号で一致したとき、それに応じて保持されているトークン種 (type) とトークン ID (id) を取り出す。文字列が一致しなかったとき、そのトークンは数字・文字列・識別子のいずれかであるから、switch 文の default で、渡された文字列が数字であるか調べる isUnsignedInt(char) メソッドと、文字列であるか調べる isPascalString(char) メソッドによってトークン種を判別する。そのどれでもないときは識別子と判定することで、45 通りのトークン種に分類することができる。

プログラム 2: Token.java

```
1 public class Token {
2     public Token(final String name) {
3         this.name = name;
4         switch (name) {
5             case "and":
6                 this.type = "SAND"; this.id = 0; break;
7             // 省略
8             default:
9                 if (isUnsignedInt(name)) {
```

```

10         this.type = "SCONSTANT"; this.id = 44;
11     } else if (isPascalString(name)) {
12         this.type = "SSTRING"; this.id = 45;
13     } else {
14         this.type = "SIDENTIFIER"; this.id = 43;
15     }
16     break;
17 }
18 }
19 }

```

4 考察や工夫点

4.1 設計の工夫点

1. **DFA による終端判定の一元化**：先頭文字で記号・数値・文字列・英数字の 4 系統へ分岐し，各系統を DFA で処理することで分岐数を抑えた．最長一致は必要時のみ 1 文字先読みで実現し，失敗時は即 1 文字へフォールバックするように設計した．
2. **責務分離と不変データ**：Lexer は I/O と制御，Tokenizer は走査とトークン構築，Token は同定を担当させた．Token は不変として生成時に種別と ID を確定し，後工程の副作用依存を排した．
3. **表駆動の同定**：予約語・記号は表引きで明示列挙し，未一致のみ整数・文字列・識別子の順で軽量判定する構造とした．表は実装依存を局所化するための拡張点でもある．
4. **線形時間性**：各文字は高々定数回の先読みと状態遷移で確定し，計算量は入力長 n に対して $O(n)$ である．補助記憶は現在構築中トークンに対する $O(L)$ （最長トークン長）に限定した．

4.2 今後の改善点・拡張性

今後は，コメント記法（`{...}`，`(*...*)`，`//`）の正式対応と未閉じ検出を実装し，診断は行・桁情報を含む分かりやすいメッセージへ拡充する．さらに，数値リテラルを実数・指数・基数表記へ広げ，DFA を誤判定の起きにくい形へ整理して堅牢性を高める．予約語・記号表は JSON 等へ外部化してビルド時に生成することで，仕様変更や課題ごとの差分に強い構成とする．あわせて，識別子の Unicode 対応として正規化と字種判定を導入し，多言語環境でも一貫して解析できる字句体系へ発展させることで，より完成度の高い字句解析が実現できると考えられる．

5 感想

字句解析器の実装を通じて，最長一致の実装テクニックや DFA に落とし込む設計思考，そして責務分離の重要性を実感した．特に，Tokenizer の先読みは容易にバグの温床になりうるため，「必要なときだけ最小限先読みし，失敗したら即座に 1 文字確定へフォールバックする」方針が有効だった．今後はコメントや実数表記など仕様を広げても，Token の不変設計と Lexer・Tokenizer の分離により安全に拡張できる見通しが立った．コンパイラ前段の作り分けが後段（構文解析・意味解析）の開発効率を左右することを体感でき，有意義な課題であった．